

05_mcp_acp_protocols

MCP + ACP + Tool-Calling + “OKF” — Integration Research for the HelixCode → HelixAgent → HelixLLM Stack

Doc type: Deep multi-angle research report (§11.4.8 / §11.4.99 / §11.4.150) **Author:** Deep-research subagent (T1/main) **Access / verification date:** 2026-07-06 **Scope:** How to integrate Model Context Protocol (MCP), an “ACP” agent protocol, tool/function-calling normalization, and Google’s “OKF” into the Helix gateway stack so that **local + cloud models expose these capabilities** and **Claude Code / HelixCode auto-recognize them**. **Evidence discipline:** every non-obvious claim carries a cited URL. Items that could not be confirmed from primary sources are marked **UNCONFIRMED** or **NEEDS-CLARIFICATION** per §11.4.6 (no-guessing).

0. Grounding facts (given, not re-verified here)

- **HelixLLM** already serves **OpenAI-compatible** (/v1/chat/completions, /v1/models, /v1/embeddings) and **Anthropic-compatible** (/v1/messages) endpoints.
 - **HelixAgent** has its own OpenAI/Anthropic/Google-compatible REST server (internal/handlers/openai_compatible.go).
 - Existing .mcp.json wiring is **partly rotted**: CodeGraph MCP uses macOS-absolute paths (fails on this Linux host), OpenDesign MCP is disabled. (These are wiring bugs to fix in the integration work, not research unknowns.)
-

1. MCP — Model Context Protocol (2026 state)

1.1 Spec version + cadence

- **Current stable:** 2025-11-25. **Release candidate 2026-07-28** was locked on 2026-05-21 and is scheduled to publish 2026-07-28; it is described as “the largest revision of the protocol since launch.” ([MCP blog — 2026-07-28 RC](#), accessed 2026-07-06)

- MCP is versioned by **date strings**. Known versions in the wild: 2024-11-05, 2025-03-26, 2025-06-18, 2025-11-25, 2026-07-28. ([go-sdk README](#), accessed 2026-07-06)

1.2 Transports

- **stdio** — local, same-machine; JSON-RPC 2.0 over stdin/stdout. Best for CodeGraph / OpenDesign / local tool servers a client launches as a subprocess.
- **Streamable HTTP** — remote / multi-client over HTTP(S); client → server via HTTP POST/GET, server can stream via Server-Sent Events (SSE). This is the **current** HTTP transport (it replaced the older “HTTP+SSE” two- endpoint transport from 2024-11-05). ([Transports spec 2025-03-26](#); [Auth0 — why MCP moved off SSE](#), both accessed 2026-07-06)
- **Direction of travel (2026-07-28 RC): the protocol becomes stateless at the transport layer.** The initialize/initialized handshake and the Mcp-Session-Id header are eliminated; “any MCP request can land on any server instance,” so a server can sit behind a plain round-robin load balancer. Applications keep state through **explicit handles passed between tool calls** rather than protocol-managed sessions. Six SEPs implement this. ([MCP 2026-07-28 RC](#), accessed 2026-07-06) → **Design implication:** build the HelixLLM/HelixAgent MCP-gateway HTTP surface **stateless-first** so it is forward-compatible with 2026-07-28.

1.3 The Go SDK situation (load-bearing for a Go stack)

- **There is now an official Go MCP SDK:** [github.com/modelcontextprotocol/go-sdk](#), maintained in collaboration with Google. This resolves the prior “no official Go SDK” gap. ([go-sdk repo](#), accessed 2026-07-06)
- **Latest release seen: v1.6.1 (2026-05-22);** 26 releases, 2,000+ dependents. Supports spec 2026-07-28 plus 2025-11-25, 2025-06-18, 2025-03-26, 2024-11-05. It provides both **client and server**, tool definitions, and StdioTransport / CommandTransport. ([go-sdk repo](#), accessed 2026-07-06)
 - One secondary source claims **v1.7.0+ supports 2026-07-28** and that stable client-side OAuth lands in ~v1.5.0 (end of March 2026). The exact minor-version → feature mapping is **UNCONFIRMED** against the repo release notes (repo page rendered v1.6.1 as latest at access time); pin the version explicitly and read go-sdk/releases before wiring. ([socket.dev overview](#), accessed 2026-07-06)
- **Streamable-HTTP transport in the Go SDK:** the SDK README excerpt fetched did not explicitly enumerate a Streamable-HTTP transport type (it named Stdio/Command). Treat “Go SDK ships a production Streamable-HTTP server transport” as **UNCONFIRMED** — **verify in go-sdk/mcp package docs**

before committing the remote- gateway design to it. ([pkg.go.dev go-sdk/mcp](https://pkg.go.dev/go-sdk/mcp), accessed 2026-07-06)

- **Mature third-party fallback:** github.com/mark3labs/mcp-go — widely used Go MCP implementation, useful as a cross-check or interim if an official-SDK feature is missing. ([mark3labs/mcp-go](https://mark3labs.com/mcp-go), accessed 2026-07-06)

1.4 How a model-serving gateway exposes MCP

Two distinct roles — **do not conflate them**:

1. **Gateway as MCP server** (gateway → agents): HelixLLM/HelixAgent expose their own capabilities (repomap, memory, verifier, CodeGraph proxy, project tools) as MCP tools/resources over Streamable HTTP, so any MCP client (Claude Code, Zed, etc.) can consume them.
2. **Gateway as MCP host/client** (gateway → downstream MCP servers): the gateway launches/connects to downstream MCP servers (CodeGraph, OpenDesign, filesystem) and makes their tools available to the model it is serving. This is the “**remote MCP servers as built-in tools**” pattern OpenAI added to its Responses API in 2025 — the server-side model calls MCP tools without the client wiring each one. ([OpenAI — new tools for building agents](#), accessed 2026-07-06)

Making a local model reliably drive MCP tools: MCP tools are just JSON-Schema-typed functions. The reliability problem is the model emitting schema-valid tool calls — solved with **constrained / grammar-guided decoding** (see §4). The gateway translates each MCP tools/list entry into the model’s native tool-format, runs constrained decoding on the arguments, then routes the resulting call back over MCP.

1.5 Auth + security

- MCP aligns authorization with **OAuth 2.0 / OpenID Connect**. The 2026-07-28 RC hardens this: clients must validate the iss parameter (RFC 9207), declare OIDC application_type at registration, bind credentials to a specific authorization server, and clarify refresh-token/scope accumulation. ([MCP 2026-07-28 RC](#), accessed 2026-07-06)
 - Moving off the SSE two-endpoint model to Streamable HTTP **reduces attack surface** (fewer long-lived server- push channels, standard request/response gateway inspection). ([Auth0](#), accessed 2026-07-06)
 - For the Helix stack: gate the **remote** MCP HTTP surface behind the existing auth (JWT/OAuth per stack config); keep **studio** MCP servers (CodeGraph, OpenDesign) local-only and never exposed to the network. Secrets live in .env (\$CONST-042), never in .mcp.json.
-

2. “ACP” — three different protocols share the acronym (disambiguation REQUIRED)

This is a genuine naming collision. Three unrelated 2025 protocols are all called “ACP” or “A2A.” Pick deliberately.

2.1 Agent Client Protocol (ACP) — Zed Industries — the one that fits an editor/agent gateway

- **What:** “the LSP for AI coding agents” — standardizes communication between **code editors (clients)** and **coding agents**, so any agent runs inside any supporting editor. ([Zed ACP](#); [agentclientprotocol/agent-client-protocol](#), accessed 2026-07-06)
- **Created:** August 2025 by Zed Industries, co-maintained with JetBrains; now community-governed at [github.com/agentclientprotocol](#), Apache-2.0. ([PromptLayer — ACP is the LSP for coding agents](#), accessed 2026-07-06)
- **Wire:** **JSON-RPC 2.0 over stdio**. Stable **protocol version 1** (negotiated via `protocolVersion` at init, independent of SDK release version). ([ACP repo](#), accessed 2026-07-06)
- **SDKs:** Rust (`agent-client-protocol`), TypeScript (`@agentclientprotocol/sdk`), Python, Java, Kotlin. ([ACP repo](#), accessed 2026-07-06)
- **Ecosystem (June 2026):** ~50 agents (Claude Code, Gemini CLI, Codex, Copilot, Goose...); editors Zed (native), JetBrains (native), Neovim + Emacs (community); OpenCode and Kiro added ACP client support. ([Morph — ACP explained](#); [OpenCode ACP docs](#); [Kiro ACP docs](#), accessed 2026-07-06)
- **Relationship to MCP:** complementary, orthogonal axis. **MCP = agent ↔ tools/data**. **ACP = editor ↔ agent**. A HelixCode agent can *be* an ACP agent (so Zed/JetBrains/Neovim drive it) while *using* MCP to reach tools.

2.2 Agent Communication Protocol (ACP) — IBM Research / BeeAI — enterprise agent ↔ agent

- **What:** REST-native, async-first agent-to-agent messaging (multipart MIME, streaming), with offline/air-gapped agent discovery. Client-server architecture (not peer-to-peer). Central to IBM’s **BeeAI** platform. ([arXiv 2505.02279 — survey of interoperability protocols](#); [databooth — ACP what/why/how](#), accessed 2026-07-06)
- **Fit for the Helix gateway:** low. This is a multi-agent orchestration/registry protocol, not an editor or tool-exposure surface. Note it exists so the acronym isn’t confused.

2.3 A2A — Agent-to-Agent Protocol — Google — peer-to-peer agent tasks

- **What:** peer-to-peer task outsourcing via capability-based **Agent Cards**; built on SSE, HTTP(S), JSON-RPC, OAuth 2.0. Enterprise multi-agent workflows. ([getstream — top AI agent protocols 2026](#); [arXiv 2505.02279](#), accessed 2026-07-06)
- **Fit for the Helix gateway:** medium-low for the *editor recognition* goal; relevant later if HelixAgent federates to *other* agents. Its **Agent Card** capability-advertisement idea (§5) is worth borrowing.

2.4 Verdict on “which ACP”

The ACP that matches “so Claude Code / editors auto-recognize the agent” is the **Zed Agent Client Protocol (§2.1)**. IBM ACP (§2.2) and Google A2A (§2.3) are agent ↔ agent protocols for a different layer. **Confirm with the operator which ACP was intended** before building — but the editor-integration wording in the task strongly implies Zed ACP. (§11.4.6)

3. “OKF” — verified

OKF = Open Knowledge Format, Google Cloud, **v0.1**, released **2026-06-12/13** (open-sourced spec + reference implementations + sample data). ([Google Cloud — how OKF can improve data sharing](#); [MarkTechPost — OKF introduced](#), accessed 2026-07-06)

- **What it is:** a **vendor-neutral knowledge-packaging format** — a directory of **Markdown files with YAML frontmatter** (one required type field), with **Markdown links forming a knowledge graph**. Shippable in git, readable on GitHub, consumable by any agent, **no mandatory SDK**. It formalizes Karpathy’s “LLM wiki” pattern. ([Google Cloud blog](#); [marktechpost](#), accessed 2026-07-06)
- **What it is NOT:** it is **not a tool/agent/transport protocol** and **not a competitor to or component of MCP**. The Google Cloud announcement does not mention MCP. OKF answers “what does curated agent knowledge *look like*”; MCP answers “how does an agent *reach* tools/data.”

Honest verdict (§11.4.6): In the task’s framing “integrate MCP + ACP + tool-calling + OKF so models expose these **capabilities**,” **OKF is category-mismatched** — it is not a capability a *model/gateway* advertises or “drives” the way MCP tools or ACP sessions are. OKF’s real role in the Helix stack is **content**: it is the recommended on-disk format for the CodeGraph/RAG/memory knowledge the gateway already

serves (and it aligns neatly with the constitution’s own markdown-doc discipline). It can be **exposed *through* MCP** (an MCP resources server that serves an OKF directory) but it is not itself an integration protocol. → **Recommendation:** treat OKF as an optional **knowledge-source format for the RAG/Skills capability**, not as a fourth protocol to implement. **NEEDS-CLARIFICATION** if the operator meant something else by “OKF.”

4. Tool / function-calling normalization (one local model, many agent clients)

4.1 The three surfaces the gateway must speak

Surface	Tool definition shape	Tool-call return shape
OpenAI Chat Completions / Responses	<code>tools: [{type: "function", function: {name, parameters: <JSON Schema>}}]; strict: true recommended</code>	<code>tool_calls[]</code> with JSON-string arguments
Anthropic Messages	<code>tools: [{name, input_schema: <JSON Schema>}]</code>	<code>tool_use content block</code> with structured input
MCP	<code>tools/list → {name, inputSchema: <JSON Schema>}</code>	<code>tools/call</code> params

Sources: [OpenAI function calling](#); [vLLM tool calling](#), accessed 2026-07-06. Anthropic tool-use shape per the vLLM/tooling comparison (“Anthropic passes tools as an array similar to OpenAI but with a different return mechanism — content blocks”), same access date. **All three are JSON-Schema-centric**, which is what makes a single normalization layer feasible.

4.2 Local-model tool formats + constrained decoding (the reliability engine)

- Serving engines (vLLM, and equivalents) implement **per-model tool parsers**: `hermes`, `llama3_json`, `llama4_pythonic`, `mistral`, `qwen3_xml`, `deepseek_v3`, `granite4`, `kimi_k2`, `glm45`, etc. Qwen2.5/Qwen3 use **Hermes-style** tool tokens; the gateway must select the right parser per served model. ([vLLM tool calling](#); [Qwen function calling](#), accessed 2026-07-06)

- **Constrained / grammar-guided decoding** is the anti-bluff mechanism: when the model chooses a tool, token sampling is masked to the tool's **JSON Schema** (integer params can only emit integer tokens, etc.), so arguments are schema-valid by construction. Without it, engines fall back to regex-extracting tool calls from raw text → “arguments may occasionally be malformed or violate the schema.” ([vLLM tool calling](#), accessed 2026-07-06)
- **Schema hygiene for max compatibility (OpenAI strict style, portable to all three)**: `additionalProperties: false`, mark every field required, express optionals as nullable. Apply this normalization to *every* tool schema the gateway emits, regardless of origin. ([vLLM tool calling](#); [OpenAI function calling](#), accessed 2026-07-06)

4.3 Normalization design

Adopt a **canonical internal tool representation** = {name, description, json_schema} (JSON Schema is the lingua franca). At the edges: - **Ingest** tools from OpenAI-style tools, Anthropic tools, and MCP tools/list → canonical form. - **Dispatch** to the served model via its **native parser** (Hermes/Qwen/Llama/etc.) with **constrained decoding against the canonical schema**. - **Emit** the result back in whichever surface the client used (OpenAI tool_calls / Anthropic tool_use / MCP tools/call response).

This is exactly the LocalAI / vLLM pattern generalized: one model, schema-constrained, many client dialects. ([LocalAI OpenAI functions](#), accessed 2026-07-06)

5. Capability advertisement so Claude Code + HelixCode auto-recognize capabilities

Goal: map the CONST-040 capability flags (**MCP / LSP / ACP / Embedding / RAG / Skills / Plugins**) onto surfaces clients already poll, sourced from LLMsVerifier (CONST-036/040), never hardcoded.

Three complementary advertisement channels:

1. **OpenAI /v1/models + per-model metadata (for HelixCode's model list & CONST-040 flags)**. The base /v1/models object is thin, but the ecosystem convention is to attach a **capabilities object** to each model entry (function-calling / tools / vision / etc.). HelixLLM should extend each /v1/models entry with a capabilities block carrying the CONST-040 flags sourced from VerificationResult, so HelixCode's handleListModel reads them directly. ([OpenAI models API](#); [OpenAI compare models](#), accessed 2026-07-06) (*The precise capabilities sub-schema is a HelixLLM-owned extension — no*

single cross-vendor standard exists; **UNCONFIRMED** that any external standard dictates its shape, so define it in LLMsVerifier and document it.)

- 2. **MCP server/discover / capability negotiation (for the MCP flag + tool/resource inventory).** The 2026-07-28 RC adds a server/discover method letting clients fetch server capabilities up front, and negotiates extensions via reverse-DNS identifiers in an extensions map on client+server capabilities. HelixAgent’s MCP server surface should advertise its tool/resource inventory here; Claude Code discovers it automatically once the server is listed in .mcp.json. (MCP 2026-07-28 RC, accessed 2026-07-06)
- 3. **ACP initialize capability exchange (for the ACP/editor flag).** ACP negotiates protocolVersion and capabilities at initialize; an editor (Zed/JetBrains) that connects to HelixAgent-as-ACP-agent auto-discovers what the agent can do. Borrow A2A’s Agent Card idea for a static, cacheable capability manifest if a discovery endpoint is wanted. (ACP repo; A2A Agent Cards, accessed 2026-07-06)

Auto-recognition mechanics for Claude Code / HelixCode specifically: - Claude Code recognizes MCP servers via .mcp.json (stdio or HTTP). **Fix the rotted entries first** — replace CodeGraph’s macOS-absolute path with a PATH-resolved command (§11.4.78), re-enable OpenDesign, and make paths host-portable — otherwise “advertisement” is moot because the client can’t launch the server. - HelixCode’s model list reads capability flags from /v1/models (channel 1); its agent surface is discovered via MCP (channel 2) and/or ACP (channel 3).

6. Protocol comparison table

Protocol	Layer / question it answers	Wire / transport	Version (2026-07-06)	Go support
MCP (Anthropic + community)	agent ↔ tools/data/resources	JSON-RPC 2.0 over stdio or Streamable HTTP (SSE); stateless in 2026-07-28	stable 2025-11-25; RC 2026-07-28	Official modelcontextprotocol/go-sdk (w/ Google), v1.6.1; 3rd-party mark3labs/mcp-go
ACP — Agent Client Protocol (Zed/JetBrains)	editor ↔ agent	JSON-RPC 2.0 over stdio	protocol 1	Rust/TS/Py/Java/Kotlin SDKs; no first-party Go

Protocol	Layer / question it answers	Wire / transport	Version (2026-07-06)	Go support
				SDK found (UNCONFIRMED)
ACP — Agent Communication Protocol (IBM/ BeeAI)	agent ↔ agent (enterprise)	REST, async, multipart MIME	2025 (BeeAI)	n/a
A2A (Google)	agent ↔ agent (peer task outsourcing)	SSE + HTTP(S) + JSON-RPC + OAuth2; Agent Cards	2025	Google-backed SDKs
OKF — Open Knowledge Format (Google Cloud)	knowledge packaging (not a protocol)	Markdown + YAML files in git	v0.1 (2026-06-12)	format only, no SDK needed
Tool/function calling (OpenAI/ Anthropic/local)	model ↔ tool invocation	JSON- Schema tool defs; constrained decoding	ongoing	vLLM-class parsers (hermes/qwen/...)

7. Recommended integration design for the Helix gateway

Layering (bottom → top):

1. **Canonical tool layer (JSON Schema).** Internal {name, description, json_schema} representation. Normalize every tool schema to OpenAI-strict style (additionalProperties:false, all-required, nullable optionals). Enforce **constrained decoding** for local models against these schemas — this is the anti-bluff guarantee that a local model's tool calls are schema-valid.

2. MCP layer (PRIMARY) — Go, official SDK.

- *Server role:* HelixAgent/HelixLLM expose Helix tools+resources over **Streamable HTTP** (stateless-first, forward-compatible with 2026-07-28) and over **stdio** for local clients. Advertise via `server/discover`.
- *Host role:* the gateway launches downstream MCP servers (CodeGraph, OpenDesign, filesystem) and folds their `tools/list` into the canonical tool layer, so the served model can call them.
- Use `github.com/modelcontextprotocol/go-sdk` (pin the version; **verify Streamable-HTTP server transport presence in the package docs before committing** — §1.3). Keep `mark3labs/mcp-go` as a documented fallback.

3. ACP layer (SECONDARY, editor recognition) — Zed Agent Client Protocol.

- Make HelixAgent implement ACP **protocol v1** (JSON-RPC over stdio) so Zed / JetBrains / Neovim / OpenCode / Kiro discover and drive it as a native external agent.
- **Go SDK gap:** no first-party Go ACP SDK was found (**UNCONFIRMED**); options are (a) implement the small ACP v1 JSON-RPC surface directly in Go (it is LSP-sized), or (b) bridge via the TS/Rust SDK. Prefer (a) for a Go stack; extend the catalogue submodule rather than vendoring (§11.4.74).

4. Capability advertisement layer (CONST-040). Source MCP/LSP/ACP/Embedding/RAG/Skills/Plugins flags from LLMsVerifier VerificationResult; surface them on (i) `/v1/models[].capabilities` for HelixCode's model list, (ii) `MCP server/discover`, (iii) `ACP initialize`. No hardcoded flags (CONST-036/040).

5. OKF as content, not protocol. Store curated agent knowledge (schemas, runbooks, metrics) as an OKF directory; serve it through an MCP resources server so it lands in the RAG/Skills capability. Do **not** model OKF as a fourth protocol.

Immediate hygiene fixes (prerequisite to any advertisement working): repair `.mcp.json` — host-portable PATH-resolved CodeGraph command (drop macOS-absolute path), re-enable OpenDesign, keep stdio MCP servers local-only, secrets in `.env`.

8. Top risks

1. **Go SDK feature/version drift (MCP + ACP).** The MCP Go SDK's exact minor-version → feature map (esp. Streamable-HTTP server transport and stable

OAuth) is **UNCONFIRMED** from primary release notes at access time, and **no first-party Go ACP SDK was found**. → Pin versions, read go-sdk/releases + go-sdk/mcp package docs before wiring, and budget for a hand-rolled Go ACP v1 surface.

2. **2026-07-28 statelessness churn**. The RC removes the initialize handshake and Mcp-Session-Id and moves to explicit handles. Building against 2025-11-25 session semantics risks a near-term rewrite. → Design the HTTP gateway stateless-first now; treat sessions as application handles, not transport state.
3. **“ACP” / “OKF” category errors**. Three protocols share “ACP” (Zed editor-agent vs IBM agent-agent vs Google A2A), and “OKF” is a *knowledge format*, not a capability/protocol. Building the wrong ACP, or treating OKF as a protocol to “expose,” wastes a cycle. → Operator confirmation on intended ACP; NEEDS-CLARIFICATION on OKF’s intended role (§11.4.6).

(Secondary risk: local-model tool-format fragmentation — each model needs the correct parser (hermes/qwen/...) and constrained decoding, or tool calls silently malformed. Mitigated by the canonical-schema + grammar layer in §7.1.)

Sources verified 2026-07-06

- MCP 2026-07-28 RC — <https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/>
- MCP Transports spec (2025-03-26) — <https://modelcontextprotocol.io/specification/2025-03-26/basic/transports>
- Auth0 — MCP Streamable HTTP / off-SSE — <https://auth0.com/blog/mcp-streamable-http/>
- Official Go MCP SDK repo — <https://github.com/modelcontextprotocol/go-sdk>
- Go MCP SDK package docs — <https://pkg.go.dev/github.com/modelcontextprotocol/go-sdk/mcp>
- socket.dev — official Go SDK overview — <https://socket.dev/blog/official-go-sdk-for-mcp>
- mark3labs/mcp-go (third-party) — <https://github.com/mark3labs/mcp-go>
- Zed Agent Client Protocol — <https://zed.dev/acp>
- ACP repo (community governance) — <https://github.com/agentclientprotocol/agent-client-protocol>
- PromptLayer — ACP = LSP for coding agents — <https://blog.promptlayer.com/agent-client-protocol-the-lsp-for-ai-coding-agents/>
- Morph — ACP explained (ACP vs MCP) — <https://www.morphllm.com/agent-client-protocol>
- OpenCode ACP docs — <https://opencode.ai/docs/acp/>
- Kiro ACP docs — <https://kiro.dev/docs/cli/acp/>

- arXiv 2505.02279 — survey of MCP/ACP/A2A/ANP — <https://arxiv.org/html/2505.02279v1>
- databooth — IBM ACP what/why/how — <https://www.databooth.com.au/posts/acp/>
- getstream — top AI agent protocols 2026 (A2A/Agent Cards) — <https://getstream.io/blog/ai-agent-protocols/>
- Google Cloud — Open Knowledge Format — <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing/>
- MarkTechPost — OKF introduction — <https://www.marktechpost.com/2026/06/16/google-cloud-introduces-open-knowledge-format-okf-a-vendor-neutral-markdown-spec-for-giving-ai-agents-curated-context/>
- vLLM — Tool Calling (parsers + constrained decoding) — https://docs.vllm.ai/en/stable/features/tool_calling/
- Qwen — Function Calling — https://qwen.readthedocs.io/en/latest/framework/function_call.html
- LocalAI — OpenAI Functions/Tools — <https://localai.io/features/openai-functions/>
- OpenAI — Function calling guide — <https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI — Models API — <https://developers.openai.com/api/docs/models>
- OpenAI — New tools for building agents (remote MCP in Responses) — <https://openai.com/index/new-tools-for-building-agents/>

Confidence notes (§11.4.6): MCP spec/versioning, Zed ACP identity, and OKF identity are **well-corroborated by primary sources**. Marked **UNCONFIRMED**: exact Go MCP SDK minor-version → feature mapping, Go MCP SDK Streamable-HTTP server-transport presence, existence of a first-party Go ACP SDK. Marked **NEEDS-CLARIFICATION**: which “ACP” the operator intends (Zed strongly implied) and OKF’s intended role (recommended: knowledge-content format, not a protocol).